

LAPORAN TUGAS 2



Disusun oleh:

Nama : Cynthia Citra Azaria

NIM : 2410651025

Mata Kuliah:

Sistem Operasi

Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DASAR TEORI

1. Konsep Teori

Proses pada dasarnya adalah program yang hidup dan sedang berjalan . Setiap proses bertindak sebagai satuan dasar untuk mengelola pekerjaan dalam sebuah sistem operasi.

- Pengenal Proses (PID): Setiap proses memiliki nomor identifikasi yang unik atau PID (Process ID) agar sistem dapat membedakan dan melacaknya.
- Hubungan Induk-Anak (Parent-Child Process): Suatu proses dapat melahirkan atau membuat proses baru (disebut proses anak atau *child process*) biasanya melalui perintah sistem seperti `fork()`. Proses yang membuatnya disebut proses induk atau *parent process*.
- Blok Kontrol Proses (PCB): Ini adalah kartu identitas atau arsip data penting yang disimpan dalam kernel sistem operasi. PCB (Process Control Block) menyimpan seluruh informasi status dan lingkungan kerja (konteks) yang penting dari proses tersebut.
- Daur Hidup Proses: Proses melewati serangkaian tahapan atau status yang berbeda sepanjang keberadaannya (misalnya, *Baru*, *Sedang Berjalan*, *Menunggu*, *Siap*, *Selesai*).
- Pengakhiran Proses dengan `wait()`: Proses induk memiliki kemampuan untuk menanggguhkan dirinya dan menunggu hingga proses anak yang diciptakannya selesai dieksekusi, menggunakan perintah sistem `wait()`.

2. Mekanisme Komunikasi Antar Proses (IPC)

Komunikasi Antar Proses (IPC) adalah seperangkat mekanisme yang memungkinkan proses-proses yang terpisah untuk berinteraksi, bertukar data, dan menyelaraskan kegiatan mereka.

- Pipa (Pipes) - Saluran Searah: Mekanisme ini menyediakan saluran komunikasi satu arah (unidireksional) dan umumnya digunakan oleh proses-proses yang memiliki hubungan langsung (induk dan anak).
- Pipa Bernama (Named Pipes atau FIFO) - Komunikasi Dua Arah: Juga dikenal sebagai FIFO (First-In, First-Out), metode ini memungkinkan komunikasi antara proses yang tidak memiliki hubungan langsung. Pipa ini dinamai dan diperlakukan layaknya berkas dalam sistem berkas (direktori). Dapat mendukung komunikasi dua arah (bidireksional) jika sepasang pipa digunakan (satu untuk setiap arah transmisi).
- Memori Bersama (Shared Memory): Ini adalah metode komunikasi tercepat karena melibatkan berbagi segmen memori fisik yang sama di antara proses-proses yang terlibat. Mereka mengakses dan memodifikasi data pada lokasi memori yang sama secara langsung.

TUGAS

Tugas 1: Sistem Manajemen Proses

```
sintiya@LAPTOP-PPED66E9:~$ nano process_manager.c
sintiya@LAPTOP-PPED66E9:~$ nano process_manager.c
sintiya@LAPTOP-PPED66E9:~$ gcc process_manager.c -o p_manager
sintiya@LAPTOP-PPED66E9:~$ ./p_manager
Parent Process (PID: 501) is starting...
```

```
Child Process (PID: 502) for task 'Sorting' started.
Child Process (PID: 503) for task 'Searching' started.
```

```
Parent is waiting for children to finish...
```

```
Child Process (PID: 504) for task 'Calculation' started.
PID 502 (Sorting): Array sorted. First element: 11
PID 503 (Searching): Found value 60 at index 4
PID 504 (Calculation): Fibonacci number 25 is 75025
```

```
=====
                        Laporan Proses Anak
=====
Task Name      | PID      | Exec Time (ms) | Status
-----|-----|-----|-----
Sorting        | 502      | 0.120          | SUCCESS
Searching       | 503      | 0.037          | SUCCESS
Calculation     | 504      | 0.053          | SUCCESS
=====
```

```
Parent Process finished.
```

Buat file process_manager.c

```
GNU nano 6.2 process_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

// Struktur untuk menyimpan informasi proses
typedef struct {
    char task_name[20];
    pid_t pid;
    double execution_time;
    int status; // 0: Success, 1: Failure
} ProcessInfo;

// --- Fungsi Simulasi Tugas ---

// Tugas 1: Sorting (Bubble Sort)
void task_sorting() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);
    int i, j, temp;
    // Simulasi pekerjaan
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    // Cetak hasil
    printf("PID %d (Sorting): Array sorted. First element: %d\n", getpid(), arr[0]);
}

// Tugas 2: Searching (Linear Search)
void task_searching() {
    int arr[] = {10, 20, 80, 30, 60, 50, 110, 130, 170};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 60; // Nilai yang dicari
    int i;
    int found = 0;
    // Simulasi pekerjaan
    for (i = 0; i < n; i++) {
        if (arr[i] == x) {
            found = 1;
            break;
        }
    }
    // Cetak hasil
    if (found) {
        printf("PID %d (Searching): Found value %d at index %d\n", getpid(), x, i);
    } else {
        printf("PID %d (Searching): Value %d not found\n", getpid(), x);
    }
}

// Tugas 3: Calculation (Fibonacci ke-25)
void task_calculation() {
    int n = 25;
    long long a = 0, b = 1, next = 0;
    int i;
    // Simulasi pekerjaan (Menghitung Fibonacci)
    for (i = 0; i <= n; ++i) {
        if (i <= 1) {
            next = i;
        } else {
            next = a + b;
            a = b;
            b = next;
        }
    }
}
```

```
GNU nano 6.2 process_manager.c
}
// Cetak hasil
printf("PID %d (Calculation): Fibonacci number %d is %lld\n", getpid(), n, next);
}

// --- Fungsi Utama ---
int main() {
    // Array untuk menyimpan informasi dari 3 proses anak
    ProcessInfo p_info[3];
    pid_t pids[3];
    void (*tasks[3])() = {task_sorting, task_searching, task_calculation};
    char *task_names[3] = {"Sorting", "Searching", "Calculation"};
    int i;

    printf("Parent Process (PID: %d) is starting...\n\n", getpid());

    // 1. Membuat multiple child processes
    for (i = 0; i < 3; i++) {
        clock_t start_time = clock(); // Waktu mulai diukur sebelum fork

        pid_t pid = fork();

        if (pid < 0) {
            // Error
            perror("fork failed");
            exit(EXIT_FAILURE);
        } else if (pid == 0) {
            // Child Process
            printf("Child Process (PID: %d) for task '%s' started.\n", getpid(), task_names[i]);

            // Setingan waktu tidur untuk mensimulasikan waktu eksekusi yang berbeda
            int sleep_time = (i + 1) * 2; // 2, 4, 6 milidetik
            usleep(sleep_time * 1000); // usleep menerima microdetik

            // Jalankan tugas yang berbeda
            tasks[i]();
        }

        // Selesai di proses anak
        exit(EXIT_SUCCESS); // Keluar dengan status sukses
    } else {
        // Parent Process
        // Simpan PID anak dan nama tugas
        pids[i] = pid;
        snprintf(p_info[i].task_name, 20, "%s", task_names[i]);
        p_info[i].pid = pid;
    }
}

// 2. Parent process mengkoordinasi dan mengumpulkan hasil
int status;
pid_t finished_pid;

printf("\nParent is waiting for children to finish...\n\n");

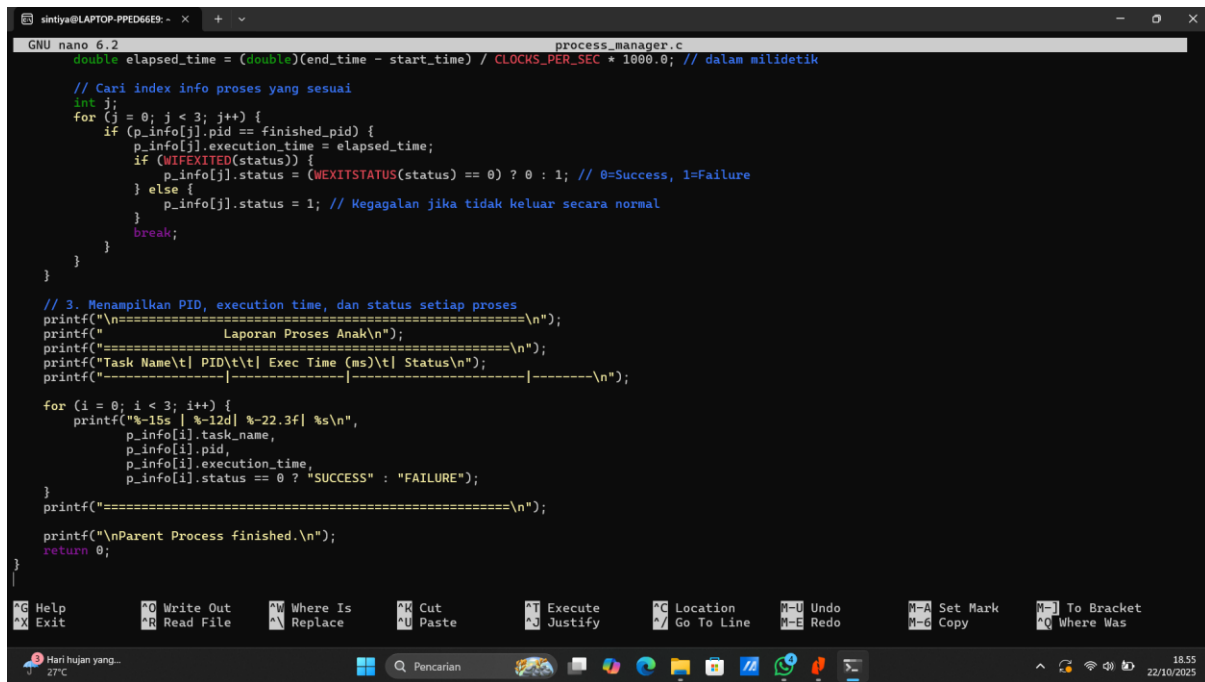
for (i = 0; i < 3; i++) {
    clock_t start_time = clock(); // Pengukuran waktu harus di sekitar waitpid

    // Menunggu proses anak tertentu
    finished_pid = waitpid(pids[i], &status, 0);

    if (finished_pid == -1) {
        perror("waitpid error");
        continue;
    }

    // Hitung waktu eksekusi
    clock_t end_time = clock();
    double elapsed_time = (double)(end_time - start_time) / CLOCKS_PER_SEC * 1000.0; // dalam milidetik

    // Cari index info proses yang sesuai
    int j;
    for (j = 0; j < 3; j++) {
        if (p_info[j].pid == finished_pid) {
            p_info[j].execution_time = elapsed_time;
        }
    }
}
```



```
GNU nano 6.2 process_manager.c
double elapsed_time = (double)(end_time - start_time) / CLOCKS_PER_SEC * 1000.0; // dalam milidetik

// Cari index info proses yang sesuai
int j;
for (j = 0; j < 3; j++) {
    if (p_info[j].pid == finished_pid) {
        p_info[j].execution_time = elapsed_time;
        if (WIFEXITED(status)) {
            p_info[j].status = (WEXITSTATUS(status) == 0) ? 0 : 1; // 0=Success, 1=Failure
        } else {
            p_info[j].status = 1; // Kegagalan jika tidak keluar secara normal
        }
        break;
    }
}

// 3. Menampilkan PID, execution time, dan status setiap proses
printf("\n=====\\n");
printf("Laporan Proses Anak\\n");
printf("=====\\n");
printf("Task Name\\t\\t PID\\t\\t Exec Time (ms)\\t\\t Status\\n");
printf("-----|-----|-----\\n");

for (i = 0; i < 3; i++) {
    printf("%-15s | %-12d | %-22.3f | %s\\n",
        p_info[i].task_name,
        p_info[i].pid,
        p_info[i].execution_time,
        p_info[i].status == 0 ? "SUCCESS" : "FAILURE");
}
printf("=====\\n");
printf("\\nParent Process finished.\\n");
return 0;
}
```

Tujuan

- **Membuat multiple child processes** : Untuk memahami dan mempraktikkan penggunaan *system call* `fork()` untuk menciptakan proses-proses baru dan menetapkan hubungan *parent-child* dalam sebuah program.
- **sorting, searching, calculation** : Untuk mendemonstrasikan bagaimana pembagian kerja (paralelisme) dapat diterapkan dalam sistem. Ini menunjukkan bahwa proses yang berbeda dapat menjalankan fungsi atau algoritma yang terisolasi secara simultan.
- **Parent process mengkoordinasi dan mengumpulkan hasil** : Untuk melatih penggunaan *system call* seperti `wait()` atau `waitpid()` pada proses *parent*. Tujuannya adalah memastikan *parent* mengelola daur hidup *child*, mencegah *zombie processes*, dan mengintegrasikan hasil dari pekerjaan yang telah diselesaikan oleh *child processes*.
- **Menampilkan PID, execution time, dan status setiap proses** : Untuk memahami konsep Process Control Block (PCB) secara fungsional. Tujuannya adalah melacak dan memonitor informasi penting dari setiap proses, seperti identitas unik (PID), performa (waktu eksekusi), dan status terminasi, yang merupakan fungsi inti dari sistem operasi.

Analisis Performa

1) Peran "Bos" (Proses Induk)

Proses utama (*Parent*) kita bertindak seperti seorang **manajer proyek**. Tugasnya cuma tiga:

- **Mendelegasikan**: Menyuruh tiga "karyawan" (*Child Processes*) untuk mengerjakan tiga tugas berbeda (Sortir, Cari, Hitung).
- **Mengawasi**: Menunggu semua karyawan selesai bekerja.

- **Mencatat Waktu:** Mengukur berapa lama persisnya waktu yang dihabiskan setiap karyawan untuk menyelesaikan tugasnya, lalu melaporkannya.
- **Pencarian (Searching):** Ini yang **paling ngebut** (0.037 ms). Wajar, karena mencari satu angka dalam daftar pendek itu pekerjaan yang paling ringan dan cepat.
- **Perhitungan (Calculation):** Sedikit lebih lambat (0.053 ms). Ini perlu mengulang perhitungan Fibonacci 25 kali, jadi perlu sedikit usaha lebih.
- **Pengurutan (Sorting):** Ini yang **paling lama** (0.120 ms). Meskipun daftarnya pendek, proses *Bubble Sort* itu butuh banyak langkah bolak-balik untuk membandingkan dan menukar posisi angka. Jadi, secara alami, ini yang paling berat dan lambat.

Tugas 2: Chat Application dengan IPC

```
sintiya@LAPTOP-PPED66E9: ~  
sintiya@LAPTOP-PPED66E9:~/praktikum_os/monitor_dir$ ./chat_app Ria  
=== SELAMAT DATANG Ria DI CHAT ROOM ===  
  
Menu:  
1. Kirim Pesan  
2. Lihat History Pesan  
3. Exit  
Pilih opsi (1-3): 1  
Ketik pesan Anda: hai  
[Pesan terkirim dari Ria]  
  
Menu:  
1. Kirim Pesan  
2. Lihat History Pesan  
3. Exit  
Pilih opsi (1-3): 1  
Ketik pesan Anda: aku lagi dengerin musik nih  
[Pesan terkirim dari Ria]  
  
Menu:  
1. Kirim Pesan  
2. Lihat History Pesan  
3. Exit  
Pilih opsi (1-3): 2  
=== HISTORY PESAN ===  
Citra> hai Citra  
Citra> kamu lagi ngapain nih?  
Citra> hai  
Ria> hai  
Ria> aku lagi dengerin musik nih  
=====
```

```
sintiya@LAPTOP-PPED66E9: ~  
sintiya@LAPTOP-PPED66E9:~/praktikum_os/monitor_dir$ nano chat_app.h  
sintiya@LAPTOP-PPED66E9:~/praktikum_os/monitor_dir$ nano chat_server.c  
sintiya@LAPTOP-PPED66E9:~/praktikum_os/monitor_dir$ gcc chat_server.c -o chat_app -std=c99  
sintiya@LAPTOP-PPED66E9:~/praktikum_os/monitor_dir$ ./chat_app Citra  
=== SELAMAT DATANG Citra DI CHAT ROOM ===  
  
Menu:  
1. Kirim Pesan  
2. Lihat History Pesan  
3. Exit  
Pilih opsi (1-3): 2  
=== HISTORY PESAN ===  
[Belum ada pesan]  
=====
```


Buat file chat_app.h

```
GNU nano 6.2
#define MAX_MESSAGES 10    // Batasan jumlah pesan yang disimpan
#define MAX_MESSAGE_LEN 128 // Batasan panjang satu pesan
#define MAX_USER_LEN 16    // Batasan panjang nama pengguna

// Struktur untuk satu pesan
typedef struct {
    char sender[MAX_USER_LEN];
    char text[MAX_MESSAGE_LEN];
} Message;

// Struktur untuk Shared Memory
typedef struct {
    int count; // Jumlah pesan saat ini
    Message history[MAX_MESSAGES]; // Array untuk menyimpan pesan
} SharedData;

// Konstanta Kunci untuk IPC (gunakan angka unik yang besar)
#define SHM_KEY 1234
#define SEM_KEY 5678
```

Buat file chat_server.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>

#include "chat_app.h"

union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

// --- FUNGSI SEMAPHORE ---
void sem_wait(int sem_id) {
    struct sembuf sb = {0, -1, 0};
    if (semop(sem_id, &sb, 1) == -1) {
        perror("semop P failed");
        exit(EXIT_FAILURE);
    }
}

void sem_signal(int sem_id) {
    struct sembuf sb = {0, 1, 0};
    if (semop(sem_id, &sb, 1) == -1) {
        perror("semop V failed");
        exit(EXIT_FAILURE);
    }
}

// --- FUNGSI UTAMA CHAT ---

void view_history(SharedData *shm_data) {
    printf("\n=== HISTORY PESAN ===\n");
    if (shm_data->count == 0) {
        printf("[Belum ada pesan]\n");
        printf("=====\n");
        return;
    }

    for (int i = 0; i < shm_data->count; i++) {
        printf("%s> %s\n", shm_data->history[i].sender, shm_data->history[i].text);
    }
    printf("=====\n");
}

void send_message(int sem_id, SharedData *shm_data, const char *username) {
    char input_buffer[MAX_MESSAGE_LEN];
    printf("Ketik pesan Anda: ");
    if (fgets(input_buffer, MAX_MESSAGE_LEN, stdin) == NULL) return;

    input_buffer[strcspn(input_buffer, "\n")] = 0;

    if (strlen(input_buffer) == 0) return;

    // --- KRITIS SECTION ---
    sem_wait(sem_id);

    int index = shm_data->count % MAX_MESSAGES;

    strncpy(shm_data->history[index].sender, username, MAX_USER_LEN - 1);
    shm_data->history[index].sender[MAX_USER_LEN - 1] = '\0';

    strncpy(shm_data->history[index].text, input_buffer, MAX_MESSAGE_LEN - 1);
    shm_data->history[index].text[MAX_MESSAGE_LEN - 1] = '\0';

    if (shm_data->count < MAX_MESSAGES) {

```

```

    if (shm_data->count < MAX_MESSAGES) {
        shm_data->count++;
    }

    printf("[Pesan terkirim dari %s]\n", username);

    sem_signal(sem_id);
    // --- AKHIR KRITIS SECTION ---
}

// --- FUNGSI MENU PENGGUNA DENGAN HAPUS OTOMATIS ---

void user_menu(int shm_id, int sem_id, SharedData *shm_data, const char *username) {
    int choice;
    printf("\n=== SELAMAT DATANG %s DI CHAT ROOM ===\n", username);

    while (1) {
        printf("\nMenu:\n");
        printf("1. Kirim Pesan\n");
        printf("2. Lihat History Pesan\n");
        printf("3. Exit\n");
        printf("Pilih opsi (1-3): ");

        if (scanf("%d", &choice) != 1) {
            while (getchar() != '\n');
            choice = 0;
        } else {
            while (getchar() != '\n');
        }

        switch (choice) {
            case 1:
                send_message(sem_id, shm_data, username);
                break;
            case 2:
                view_history(shm_data);

```

```

        break;
    case 3:
        printf("Selamat tinggal, %s.\n", username);

        printf("[*] Proses %s mencoba menghapus Chat Room (IPC_RMID)...\n", username);

        // Menghapus Shared Memory
        if (shmctl(shm_id, IPC_RMID, NULL) == -1) {
        } else {
            printf("[*] Shared Memory berhasil ditandai untuk dihapus.\n");
        }

        // Menghapus Semaphore
        if (semctl(sem_id, 0, IPC_RMID) == -1) {
            // perror("semctl IPC_RMID warning");
        } else {
            printf("[*] Semaphore berhasil dihapus.\n");
        }

        return; // Keluar dari fungsi dan proses
    default:
        printf("Ops! tidak valid. Coba lagi.\n");
    }
}

// --- FUNGSI MANAJEMEN IPC ---

int main(int argc, char *argv[]) {
    int shm_id, sem_id;
    SharedData *shm_data;
    const char *username;

    if (argc != 2) {
        fprintf(stderr, "Penggunaan: %s <NAMA_PENGGUNA>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
}

```

```

username = argv[1];

// --- 1. SETUP SEMAPHORE ---
sem_id = semget(SEM_KEY, 1, IPC_CREAT | 0666);
if (sem_id == -1) {
    perror("semget failed");
    exit(EXIT_FAILURE);
}

union semun arg;
arg.val = 1;
if (semctl(sem_id, 0, SETVAL, arg) == -1) {
    // Warning jika sudah ada, tidak masalah
}

// --- 2. SETUP SHARED MEMORY ---
shm_id = shmget(SHM_KEY, sizeof(SharedData), IPC_CREAT | 0666);
if (shm_id == -1) {
    perror("shmget failed");
    exit(EXIT_FAILURE);
}

// Melampirkan (Attach) Shared Memory
shm_data = (SharedData *)shmat(shm_id, NULL, 0);
if (shm_data == (SharedData *)-1) {
    perror("shmat failed");
    exit(EXIT_FAILURE);
}

// Inisialisasi data (dilindungi semaphore)
sem_wait(sem_id);

// Inisialisasi count jika dianggap sesi baru
if (shm_data->count < 0 || shm_data->count > MAX_MESSAGES) {
    shm_data->count = 0;
}

```

```

sem_signal(sem_id);

// --- 3. MENU PENGGUNA ---
user_menu(shm_id, sem_id, shm_data, username);

// --- 4. CLEANUP (Detaching) ---
// Melepaskan (Detach) Shared Memory saat proses keluar
if (shmdt(shm_data) == -1) {
    perror("shmdt failed");
}

return 0;
}

```

Tujuan Tugas 2: Aplikasi Chat dengan IPC

Tugas ini bertujuan untuk memberikan pemahaman dan pengalaman praktis dalam mengimplementasikan Komunikasi Antar Proses (IPC) dan sinkronisasi dalam lingkungan *multiprocess*.

Analisis Performa

- **sem_wait(sem_id)** : Memperlambat Performa: Ini sengaja memperkenalkan latensi (keterlambatan) serialisasi. Jika banyak proses mencoba mengirim pesan secara bersamaan, mereka akan diblokir dan harus menunggu giliran.
- **sem_signal(sem_id)** : Memastikan Keandalan: Meskipun memperlambat, ini wajib ada. Tanpa ini, akan terjadi *race condition* yang fatal (misalnya, `shm_data->count++` dilakukan bersamaan, menyebabkan hilangnya pesan).
- **view_history** : Fungsi ini tidak dilindungi oleh Semaphore. Proses hanya **membaca** dari *Shared Memory*.